

Physical Intelligence (π)

[Home](#) [Research](#) [Join Us](#)

$\pi_{0.6}^*$: a VLA that Learns from Experience

Published

November 17, 2025

Email

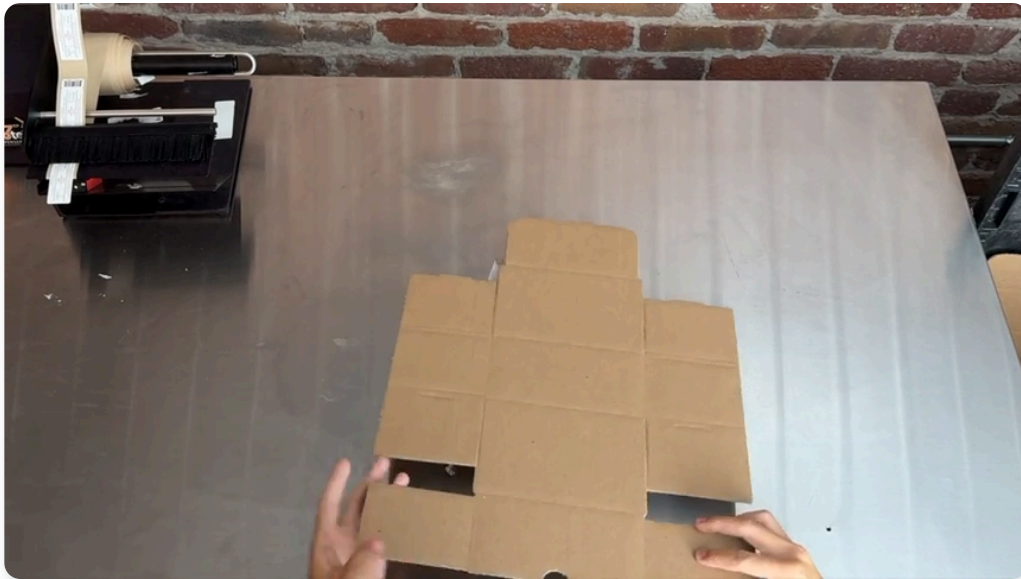
research@physicalintelligence.company

Paper

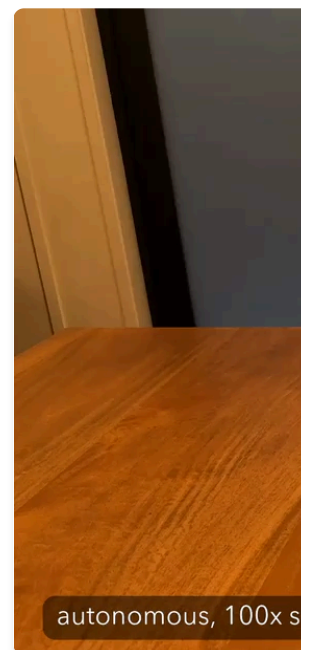
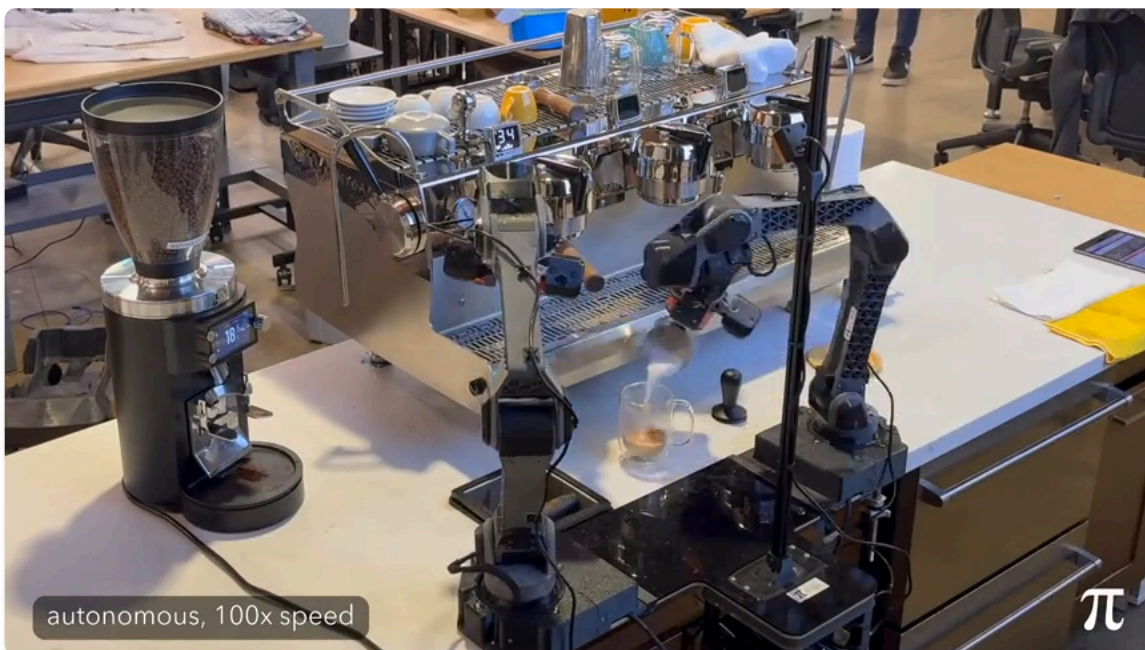
[\pi_{0.6}^*.pdf](#)



How would you learn to assemble boxes? You would like to do it quickly and efficiently, so you first ask someone to teach you the basics: what kind of strategy works well, what are the common mistakes, what is the *technique*. Second, a good teacher wouldn't just show you how it's done, but would also coach you, correcting mistakes that you make when you do it yourself. But instruction will only get you so far: in the end, practice makes perfect, and the third step to become a master box builder is to practice the task yourself enough so that it becomes second nature.



Many of the impressive results that we've seen over the past year in robotic learning use only the first step in this process, training the robot on demonstrations provided by a person. With just this step, it's not hard to get a robot to succeed half of the time, but it's very hard to get it to succeed *everytime*, much less to succeed with human-level throughput on a complex real-world task. This is a big problem, because real-world robotics tasks require a system that can work reliably and quickly. We've developed a method called RECAP (RL with Experience & Corrections via Advantage-conditioned Policies) that implements all three steps, training the robot with demonstrations, coaching it with corrections, and enabling it to improve from autonomous experience. We used RECAP to improve the latest version of our vision-language-action (VLA) model, $\pi_{0.6}$, to robustly and efficiently perform complex tasks: making espresso drinks, assembling boxes, and folding diverse laundry items. We call the resulting RL-trained model $\pi_{0.6}^*$. Training $\pi_{0.6}^*$ with RECAP on autonomous experience more than doubles the throughput on some of the hardest tasks, and can decrease failure rates by 2x or more. This enables $\pi_{0.6}^*$ to reach practically useful levels of robustness: we were able to run it to make espresso drinks for an entire day, fold novel laundry items in a new home for hours without interruptions, and assemble boxes that are used for real packaging in a factory.



After training with RECAP, $\pi_{0.6}^*$ can make various espresso drinks from 5:30am to 11:30pm, fold 50 different novel laundry items in a new home, and assemble and label 59 boxes used for packaging chocolates in a real factory, all shown here without interruption for hours. Real-time versions of each video are available [here](#).

Why is imitation not enough?

We might wonder why VLAs struggle to succeed consistently with only supervised learning (i.e., imitation), while supervised learning works so well for LLMs and other ML systems. The cause of this problem is actually [well understood](#), though a practical solution has previously proven elusive. When a VLA trained with imitation controls the robot, it will, like any model, make small mistakes – it might put the gripper in the wrong spot, miss a grasp, or knock over an object. Because the robot is interacting with a real physical environment, this mistake will produce a situation that is a bit different from situations in the training data, where the robot is more likely to make another, bigger mistake, leading to compounding errors. The small mistakes can be fixed, but the compounding errors lead to failure. This is not as big a problem for AI systems that produce a static output (like LLMs): it is specific to settings where the model is a control policy that interacts continually with an external environment, such as a robot in the real world. In practice, this means that while it's relatively easy to get VLAs to succeed at a task *some* of the time, it's quite hard to make them succeed *reliably*.

This problem could be fixed if we use additional data from the VLA's own behaviors, essentially training it to fix the mistakes that it actually makes in the real world. Just like a person can improve at a task through practice, compounding mistakes can be addressed by allowing the policy (i.e., the VLA) to practice repeatedly. But what can we use as the ground truth label for this kind of experience? If we train the policy to just copy what it did before, we would simply teach it to keep making the same mistakes. The key to enable learning from experience is to extract good training signals from bad experiential data.

Coaching with corrections and practicing with reinforcement

RECAP enables two ways to get good training signals from "bad" experiential data: *coaching* to provide corrections, where an expert shows the robot how it can fix a mistake or do better, and *reinforcement learning*, where the robot judges for itself which of its behaviors were better or worse based on the overall outcome of an episode, and iteratively learns to perform the good behaviors while avoiding the bad ones.

For coaching to be useful, an expert teleoperator needs to provide corrections showing how to recover from the mistakes that the robot actually makes in the real world. In practice, this means running our best current policy and "taking over" with manual teleoperation when the robot makes a mistake. This intervention can be used as supervision, but unlike the demonstrations used to train the original policy, the intervention provides supervision for the situations that the policy actually puts the robot into, addressing the compounding mistakes issue.



An expert teleoperator takes over and provides a real-time correction for the robot's mistake.

However, providing corrections can only take us so far: the quality of supervision from these corrections is limited by a person's ability to identify the right time to intervene and then actually provide a high-quality correction. This can work well for obvious or egregious mistakes, but for best performance – performing a task quickly, reliably, and consistently – the robot needs to learn on its own. Just like an athlete can't get far in mastering their sport if they don't practice, we also need a way for the policy to continue learning and refining the fine-grained details of its behavior from practice and *reinforcement*.

The central challenge in learning via reinforcement from task outcomes is *credit assignment*: understanding which of the actions that the robot performed caused the good outcomes, and which ones caused the bad outcomes. If the robot picks up the portafilter for an espresso machine in the wrong way, it might struggle to insert it. The mistake is not in the insertion, but in the original grasp. A correct credit assignment method would identify the grasp as a mistake, even though the failure was only experienced later.

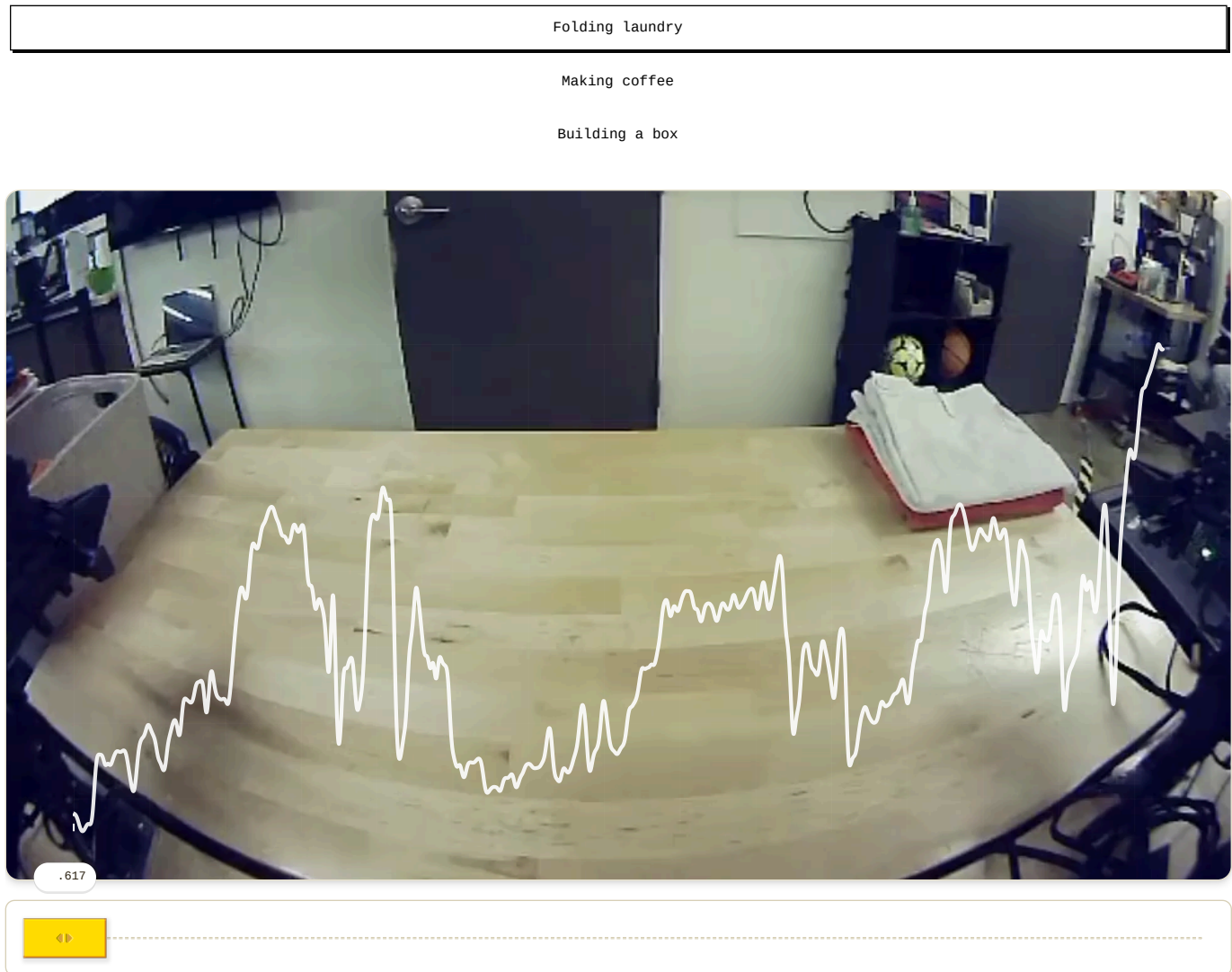


The base model trained with imitation learning struggles to insert a portafilter into an espresso machine. The mistake that led to the failure might have happened much earlier, in grasping the portafilter at the wrong angle.

Credit assignment is a key challenge in reinforcement learning. RECAP addresses this challenge by training a *value function*: a model that predicts how good a particular situation is relative to others. For example, in a game like chess, where the agent receives a reward for winning the game, the value function would predict the probability that the agent would win based on the current board state. If we can learn a value function from the robot's experience, we can determine which actions are good or bad by looking at the change in the value function: actions that result in an increase in the value function, like chess moves that lead to board states from which victory is more likely, are good actions that should be encouraged, while actions that lead to a decrease in the value should be discouraged. The illustration below shows the predictions from our value function over the course of task execution.

Value function visualization.

Drag the slider to visualize the value function at different points in an episode. A value of 1.0 corresponds to predicting that the episode has been completed successfully.



The value function predictions at different points in an episode. This value function predicts the (negative) number of steps to completion of the task. Notice that the prediction increases when the robot makes progress, and stays flat when there is little progress.

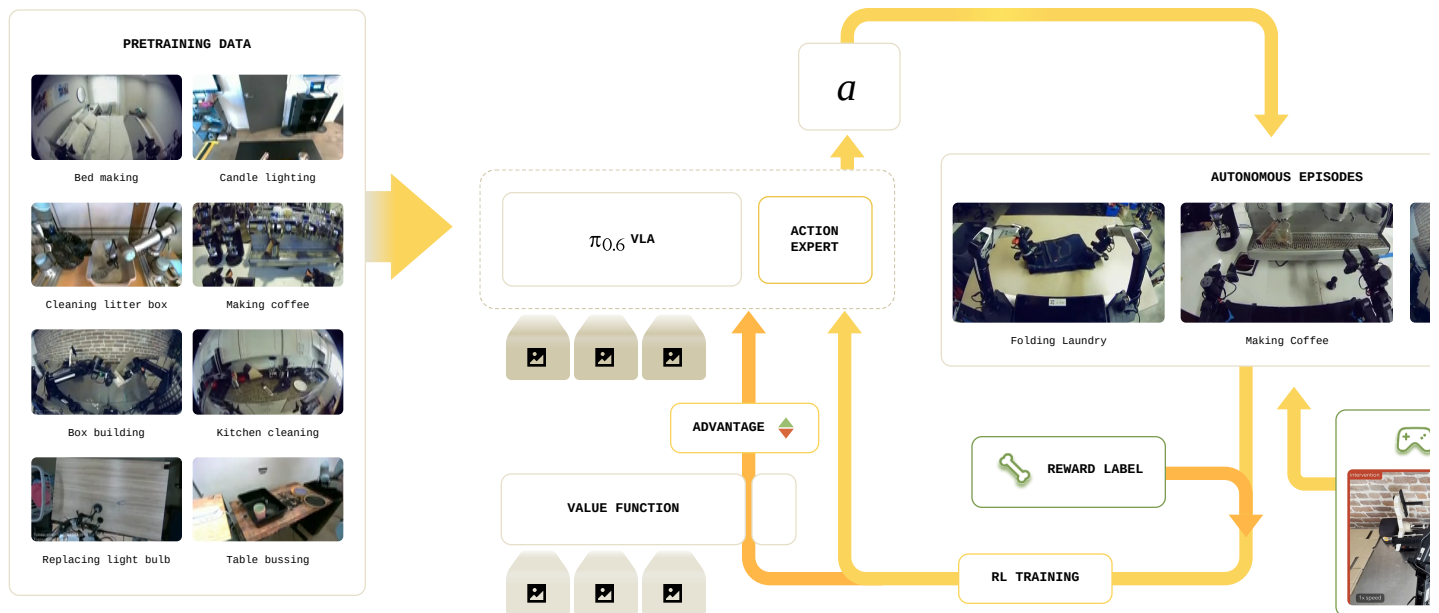
Once we've trained the value function, we need to use it to get a better policy ("policy extraction"). There are a few ways to do this, but we need a method that is scalable and can be used with large VLA models. In RECAP, we *condition* the policy (i.e., the VLA) on the change in value, using all of the data for training (both good and bad actions), while telling the VLA which actions are good or bad. Since models generalize best when provided with a lot of data, keeping all of the data in training and simply

adding the value change annotations as input is an appealing option. In RL, this "change in value" is referred to as the *advantage*. At execution time, we simply tell our advantage-conditioned VLA to perform high-advantage actions, resulting in a policy that is better than the data it was trained on.

Toward real-world tasks

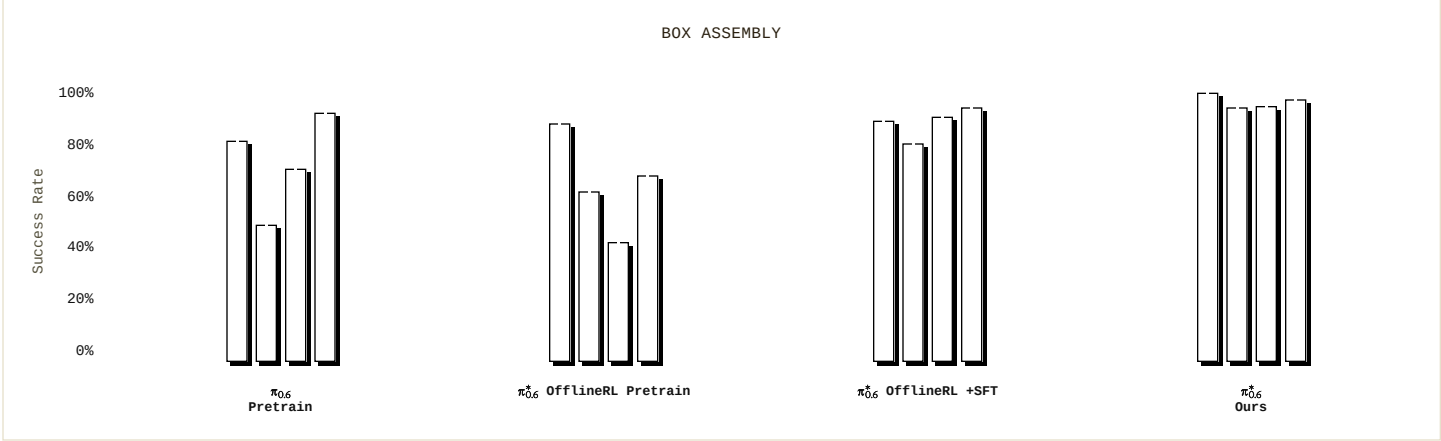
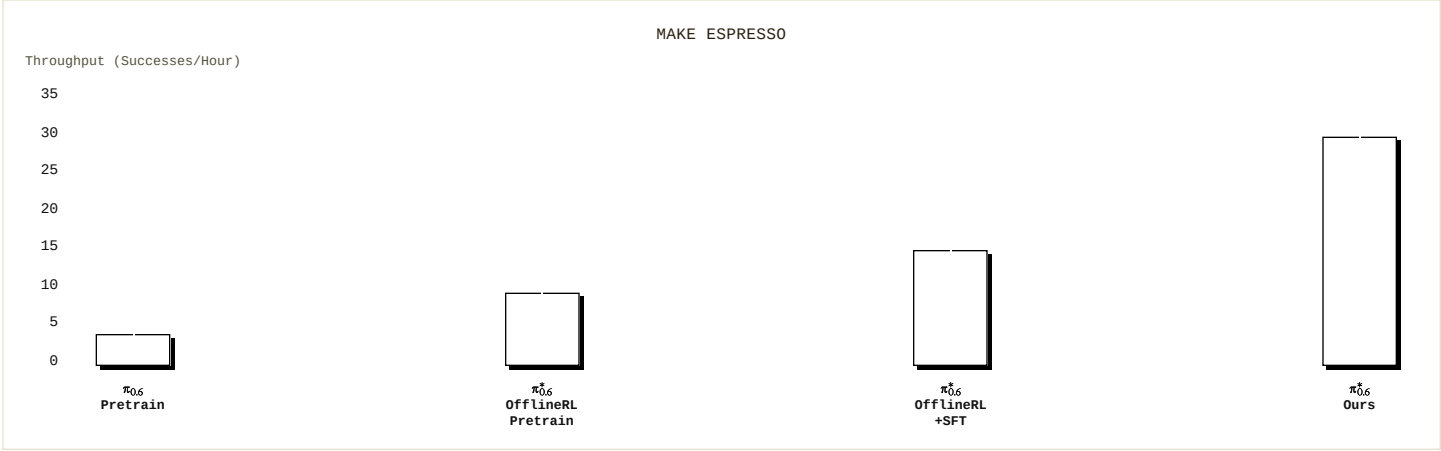
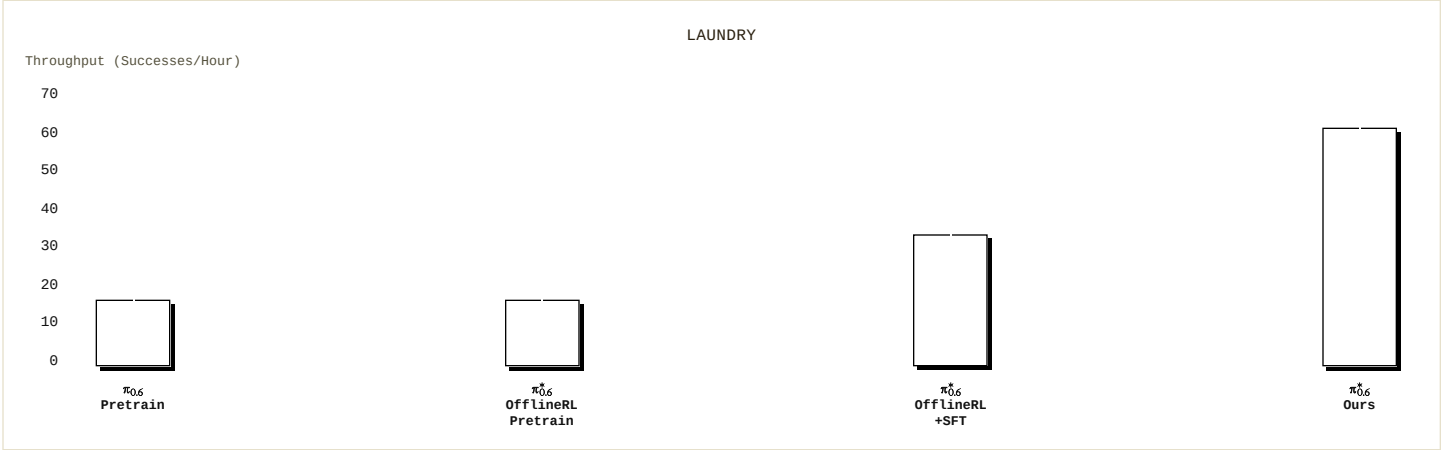
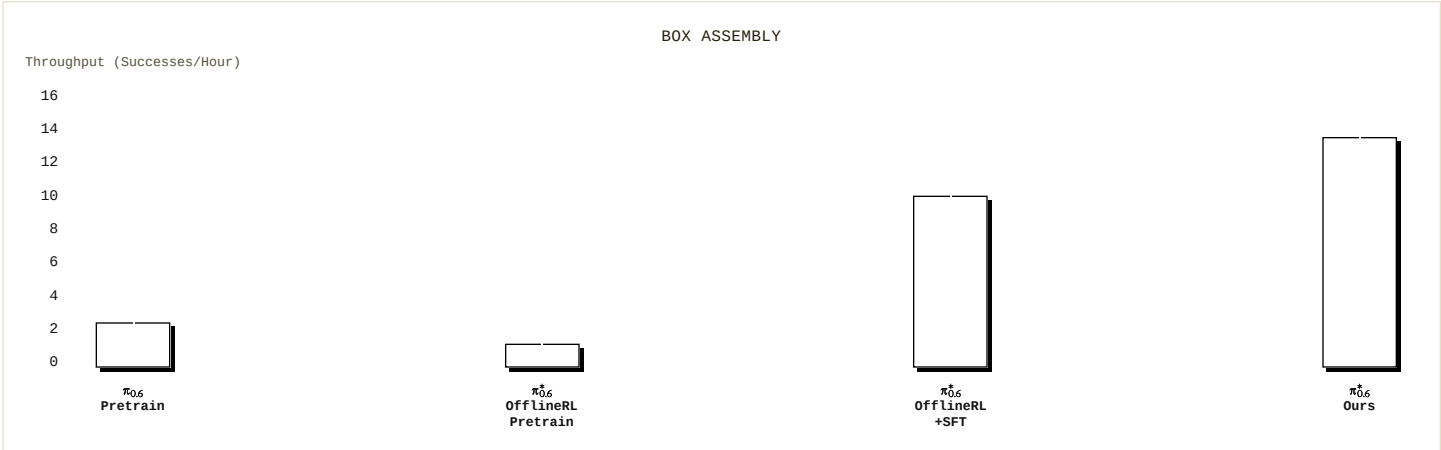
We applied RECAP to train the $\pi_{0.6}^*$ model to perform several real-world applications. $\pi_{0.6}^*$ is based on the $\pi_{0.6}$ model, which is a refinement of our earlier $\pi_{0.5}$ model, based on a slightly larger backbone and able to accommodate more heterogeneous prompts and conditioning information, as illustrated below. For a more detailed description of the $\pi_{0.6}$ model architecture, see the accompanying [model card](#).

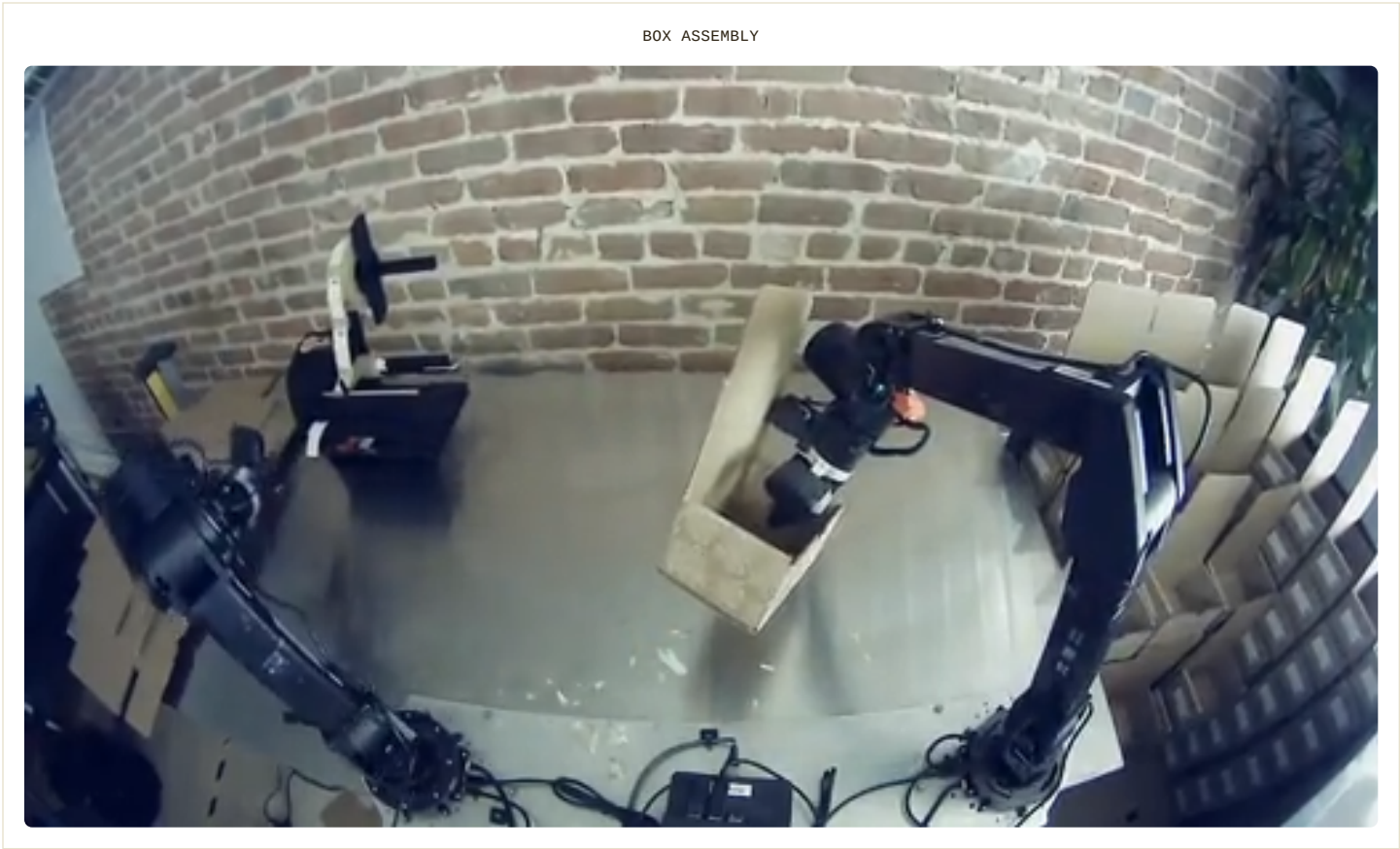
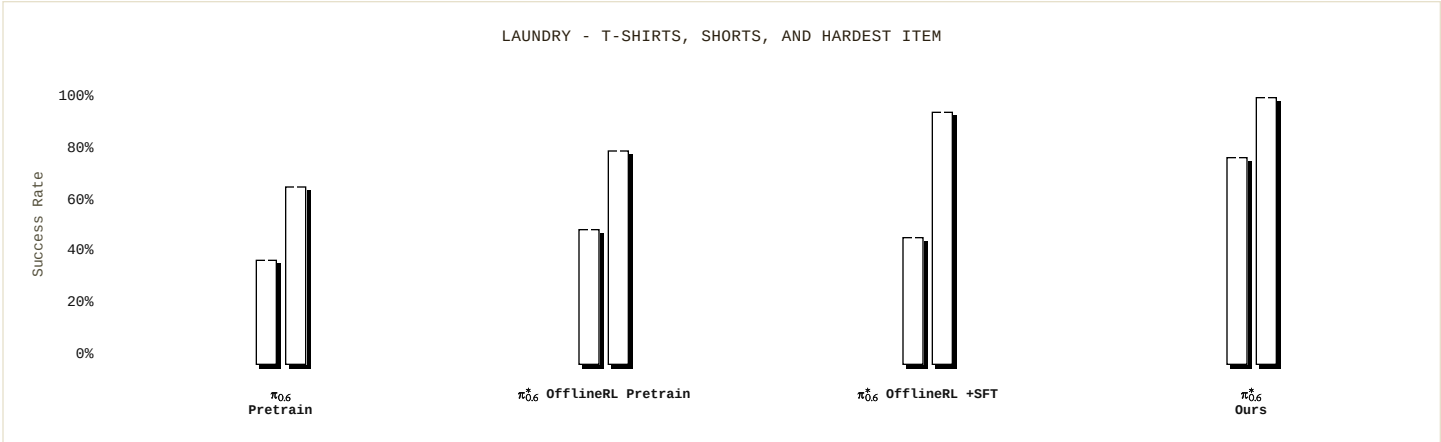
The $\pi_{0.6}$ VLA is based on a 5B-parameter vision-language model, augmented with an action expert. It supports heterogeneous prompts, including both textual commands and conditioning information, which includes annotations about the (desired) execution quality and, with RECAP, the desired action advantage.



We studied three applications: making espresso drinks, folding diverse laundry, and assembling boxes for packaging. The first stage of RECAP is to pre-train the $\pi_{0.6}^*$ model with offline RL, in contrast to the standard supervised learning method used by the base $\pi_{0.6}$ and $\pi_{0.5}$ VLAs. From here, we fine-tune $\pi_{0.6}^*$ to each task with demonstrations, and then train it further with RL using additional data collected on the robot, with both expert-provided corrections to fix large mistakes and feedback from rewards to improve from autonomous experience.

The graphs below compare the performance of the base $\pi_{0.6}$ model (trained with supervised learning), the base $\pi_{0.6}^*$ model (trained with offline RL, the first stage of RECAP), the $\pi_{0.6}^*$ model after fine-tuning to each task with demonstrations, and the final $\pi_{0.6}^*$ model finetuned to each task with on-robot experience. For each task we measure the throughput (the number of successful task completions per hour) and the success rate. Notably, we see the largest improvement on some of the hardest tasks, such as espresso making, where both throughput and success rate more than double from including on-robot experience.





LAUNDRY - T-SHIRTS, SHORTS, AND HARDEST ITEM



MAKE ESPRESSO



RECAP leads to a large improvement in throughput across all of the tasks, often with significant improvements in success rate.

Qualitatively, the final $\pi_{0.6}^*$ models were able to master each of the applications after learning from demonstration data and their on-robot experience. The videos below show some of the evaluation highlights from these tasks.

Each of the tasks has a number of challenges that make high-throughput autonomous execution challenging. The box assembly task requires performing a physically intricate behavior, folding the flaps while holding the box together. It also requires performing the task repeatedly and handling edge cases, which can be seen in the videos above: sometimes the flattened boxes stick together, such that the robot picks up multiple boxes and has to put the extra ones back, and sometimes the box needs to be refolded after a mistake. The laundry task requires handling a lot of variability and generalizing both to different initial conditions, and different laundry items. This is very difficult, because not only do different clothing items require different strategies, but different types of fabrics also have different dynamics. Finally, the espresso drinks require handling a very long horizon task, where we use a high-level language policy as with our previous [\$\pi_{0.5}\$ model](#). This task also requires pouring liquids, understanding how long to wait for the coffee grinder and espresso machine to finish, and cleaning the machine after the drink is made with a cloth towel. Each of these stages is challenging for even the best current VLA models, and $\pi_{0.6}^*$ can perform them with over a 90% success rate.

Where are we headed?

Today, robotic foundation models primarily use demonstration data that is collected manually by people (e.g., teleoperation). This makes training simple and straightforward, but presents a serious obstacle: data requires human effort, the speed and robustness of the model is limited to human-level performance, and the robot doesn't get better from experience. Methods like RECAP can in principle address these limitations by additionally learning directly from the robot's own experience. As robots are deployed more and more broadly in the real world, learning from experience might become a significant source of training data, and an essential part of the recipe for attaining high performance. Just like people learn through a combination of instruction, coaching, and practice, robots too will learn from many different data sources. But these data sources will serve distinct purposes: expert-provided demonstrations will serve to define new behaviors, coaching will serve to refine the strategy, and autonomous experience – potentially the largest source of data – will serve to perfect the behaviors, perhaps allowing them to eventually attain superhuman performance.

If you are interested in collaborating, please [reach out](#). We are particularly excited to work with companies scaling up data collection with robots deployed for real-world applications, who are looking to collaborate on autonomy.

We are also hiring! If you'd be interested in [joining us](#) please get in touch.

For researchers interested in our work, collaborations, or other queries, please write to research@physicalintelligence.company.

Physical Intelligence

[Home](#) [Research](#) [Join Us](#)